

- topologia da rede de processamento de HLG SI
- compilação do código
- Programa é executado com parâmetros de entrada ou com arquivos e saída
- Gráfico HLG é inicializado vazio
- para cada arquivo de entrada:
 - ↳ arquivo é compilado para o formato .bc
 - ↳ conteúdo do arquivo é utilizado para ser lido com biblioteca /api LLVM-C
 - ↳ linha por linha do arquivo é lida
 - ↳ para cada função executada, números são inicializados
 - ↳ funções dependentes são armazenadas em hash-map de dependências
- para cada nó novo ponteiro obtido, um nó dentro do HLG é criado, contemplando também SI

↳ continuidade workflow

(II)

↳ por cada alteração de um símbolo existente, novo nó é criado a partir do nó original, mantendo, caso necessário, as mudanças do ponteiro representado pelo S1

↳ A definição de uma ponteira deve criar um tipo de nó "objeto", representando o começo daquele valor

↳ ao encontrar um free, nó de tipo "liberação" é adicionado ao globo. Com o resto proveniente de seu objeto conhecido

↳ deve existir a possibilidade de adicionar como parâmetros de configuração do programa as funções de alocação e dealocação o qual são considerados um bloco de projeto ou código analisado

↳ apesar de existirem ferramentas capazes de identificar tais funções, este passo alonga o processo de análise

↳ continuação do workflow

(III)

↳ a árvore do Holog deve ser preenchida
obrando nós do tipo "rendição" em pontos
que precisam de função / retorno de uma
determinada função

↳ ao encontrar tal função, nós devem
ser adicionados, expandindo os lugares anteriormente
vagos

↳ este processo deve ser mantido enquanto houver
arquivos ou links a serem visitados

↳ após montagem do gráfico, é iniciado o tipo de
rede de gráfico

↳ a partir de cada nó do tipo "objeto" é realizado:

↳ visto em profundidade a procura de nós de
liberação para cada ramo do objeto

↳ ao encontrar nó de liberação, todos os nós
de tal ramificação devem ser removidos

↳ verificação dos S/S de cada nó deve ser
realizada a procura de operações involúveis

→ continuação do work flow

(IV)

→ como diferença do Holz original, um nó objeto pode ser apagado somente se todos os seus ramificações forem corretamente liberados, fazendo com que nenhumo ponteiro seja mantido

→ com isso, o possível marcar o objeto a partir de sua elocução até possível vazamento

→ após todo o ajuste, o imediato processo de análise dos nós restantes:

→ caso não haja no algum no grupo Holz, nenhum vazamento foi detectado, indicando a ausência de vazamentos

→ para nós restantes, obter links e arquivos de origem e ordenar tais dados no formato de lista, com as classificações: "must leak" e "maybe leak"

→ as links de origem de tais operações se tornam visíveis a partir de logs utilizados durante compilação

→ fim!